Course title:

Software Design Patterns (Lab)

Course code:

IS354

Program 1

Title : Implementation of OOPs concepts in Java.

Problem Description: Students can take suitable examples to implement the below concepts in Java:

1. Method overriding
2. Abstract classes
3. Interfaces

Method overriding occurs when a **subclass provides its own implementation** of a method already defined in its **superclass**.

Key Features

- Requires inheritance
- Same method name
- Same parameters
- Same return type
- Achieves runtime polymorphism

```
// Superclass
class Animal {
    void sound() {
        System.out.println("Animals make sound");
    }
}

// Subclass
class Dog extends Animal {

    // Method overriding
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

// Main class
public class Main {
    public static void main(String[] args) {

        Animal a = new Dog();   // Parent reference, child object
        a.sound();
    }
}
```

An abstract class is a class that **cannot be instantiated** and is used as a **base class**. It may contain abstract methods (without body) and concrete methods (with body).

Key Features

- Declared using **abstract** keyword
- Can contain method with or without body
- Can contain constructors and variables
- Used for partial abstraction

```
abstract class Vehicle {

    // Abstract method
    abstract void start();

    // Normal method
    void fuel() {
        System.out.println("Vehicle uses fuel");
    }
}

class Car extends Vehicle {
    void start() {
        System.out.println("Car starts using key");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle v = new Car();
        v.start();
        v.fuel();
    }
}
```

An interface is a blueprint of a class that contains **method declarations** and forces classes to implement them.

Key Features

- Achieves complete abstraction
- Supports multiple inheritance
- Methods are public and abstract by default
- Variables are public, static, and final

Interfaces enable **multiple inheritance** and **loose coupling**.

```
interface Payment {
    void pay();
}

class CreditCard implements Payment {
    public void pay() {
        System.out.println("Payment using Credit Card");
    }
}

class UPI implements Payment {
    public void pay() {
        System.out.println("Payment using UPI");
    }
}

public class Main {
    public static void main(String[] args) {
        Payment p;

        p = new CreditCard();
        p.pay();

        p = new UPI();
        p.pay();
    }
}
```